EXPERIMENT 11

Top-Down Parser for a Context-Free Grammar

Aim

To implement a simple top-down parser that checks whether a sentence can be generated by a context-free grammar.

Procedure

1. Define grammar rules for a simple sentence.
2. Create a parser that expands grammar symbols from the start symbol.
3. Match terminal symbols against input words.
4. Accept when all words and grammar symbols are consumed.
5. Test the sample sentence.

Program

```
# A compact recursive top-down parser for: S -> NP VP
grammar = {
    "S": [["NP", "VP"]],
    "NP": [["Det", "N"]],
    "VP": [["V"]],
    "Det": [["the"]],
    "N": [["cat"], ["dog"]],
    "V": [["sleeps"], ["runs"]]
}

def parse(symbol, words, pos):
    if symbol not in grammar:
        return pos + 1 if pos < len(words) and words[pos] == symbol else None
    for rule in grammar[symbol]:
        next_pos = pos
        for part in rule:
            result = parse(part, words, next_pos)
            if result is None:
                break
            next_pos = result
        else:
            return next_pos
    return None

sentence = "the cat sleeps".split()
print("Accepted" if parse("S", sentence, 0) == len(sentence) else "Rejected")
```

Output

```
OUTPUT — EXPERIMENT 11
Accepted
```

Result

Thus, the top-down parser checked the sentence against the defined context-free grammar.